

Q3 (a) Write Test Cases for Login Validation. [7 Marks]

Definition

A **Test Case** is a document that specifies the **test scenario, input data, execution steps, and expected result** to verify whether a software feature works correctly.

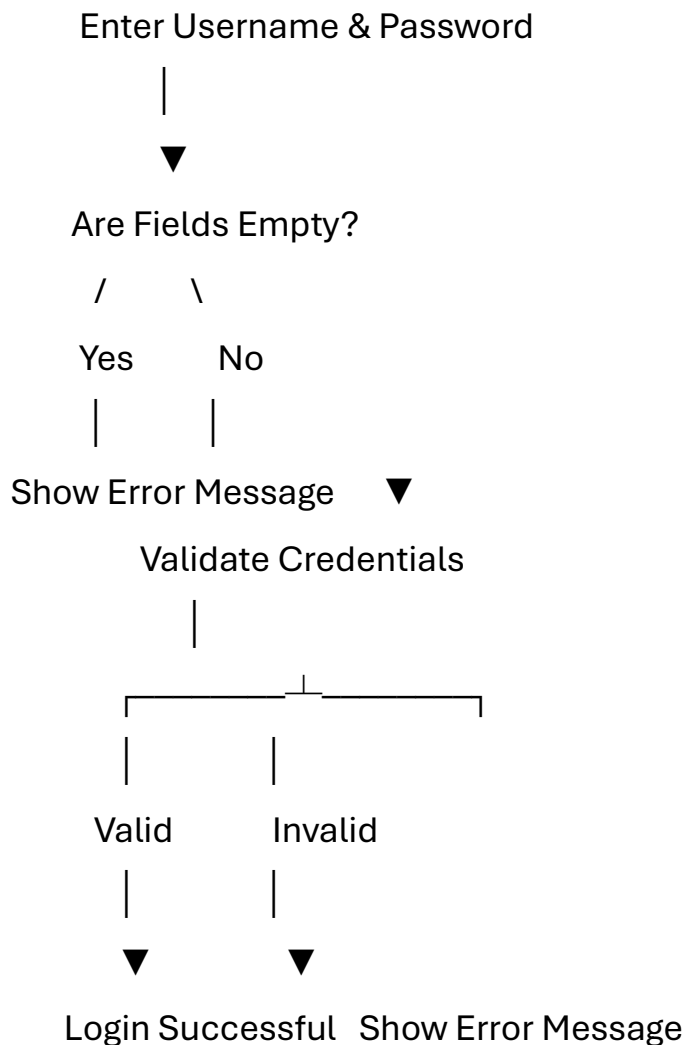
Objective: Verify the login functionality of a software application.

Test Cases for Login Validation

Test Case ID	Test Scenario	Test Data	Expected Result
TC01	Login with valid username and valid password	Username: admin, Password: admin123	User should successfully log in and be redirected to the home page.
TC02	Login with valid username and invalid password	Username: admin, Password: abc123	Display " Invalid Password " and deny login.
TC03	Login with invalid username and valid password	Username: user1, Password: admin123	Display " Invalid Username ".
TC04	Login with invalid username and invalid password	Username: user1, Password: 12345	Display " Invalid Username or Password ".
TC05	Leave username blank	Username: Blank, Password: admin123	Display " Username is Required ".
TC06	Leave password blank	Username: admin, Password: Blank	Display " Password is Required ".
TC07	Leave both username and password blank	Username: Blank, Password: Blank	Display " Username and Password are Required ".
TC08	Enter special characters	Username: @#\$%, Password: !@#	Display validation/error message.

Test Case ID	Test Scenario	Test Data	Expected Result
TC09	Password is case-sensitive	Username: admin, Password: Admin123	Login should fail if the password does not match exactly.
TC10	SQL Injection attempt	Username: ' OR '1'='1	Login should be blocked and the system should remain secure.

Login Validation Flow



Expected Validation Rules

- Username should not be blank.
- Password should not be blank.
- Username and password should match stored credentials.
- Password should be case-sensitive.

- Invalid login attempts should display appropriate error messages.
 - The system should prevent SQL Injection and unauthorized access.
-

Conclusion

Login validation testing ensures that only authorized users can access the application. Well-designed test cases help identify defects, improve security, and enhance the reliability of the login module.

Q3 (b) What are the Entry & Exit Criteria of Testing? [7 Marks]

Definition

Entry Criteria and **Exit Criteria** are predefined conditions used in the software testing process.

- **Entry Criteria** specify the conditions that must be satisfied **before testing begins**.
- **Exit Criteria** specify the conditions that must be satisfied **before testing is considered complete**.

These criteria help ensure that testing is performed in a systematic and controlled manner.

Entry Criteria of Testing

The following conditions should be met before starting testing:

1. Requirements are Available

- Software Requirement Specification (SRS) is complete and approved.

2. Test Plan is Ready

- A test plan and testing strategy have been prepared.

3. Test Cases are Prepared

- Test cases and test data are designed and reviewed.

4. Test Environment is Ready

- Hardware, software, database, and network are configured for testing.

5. Build is Available

- Developers provide a stable software build for testing.

6. Testing Tools are Available

- Required testing tools and defect tracking tools are installed.

7. Resources are Assigned

- Testers and necessary resources are available.
-

Exit Criteria of Testing

Testing is completed only when the following conditions are satisfied:

1. All Planned Test Cases are Executed

- Every planned test case has been run.

2. Critical Defects are Fixed

- All high-priority and critical defects are resolved.

3. Regression Testing is Completed

- Ensure that recent changes have not introduced new defects.

4. Test Coverage is Achieved

- Required test case coverage and requirement coverage are completed.

5. Acceptance Criteria are Met

- Software satisfies customer and business requirements.

6. Test Report is Prepared

- Testing summary, defect reports, and metrics are documented.

7. Customer/Manager Approval

- Project manager or customer approves software release.
-

Diagram

Testing Process

Entry Criteria



Test Planning





Test Execution



Defect Fixing



Regression Testing



Exit Criteria

Difference Between Entry and Exit Criteria

Entry Criteria	Exit Criteria
Checked before testing starts	Checked before testing ends
Ensures testing can begin	Ensures testing can be completed
Includes test plan, test cases, environment, and build	Includes completed testing, fixed defects, reports, and approval

Advantages

- Ensures systematic testing.
 - Reduces testing risks.
 - Improves software quality.
 - Prevents incomplete testing.
 - Helps deliver reliable software.
-

Conclusion

Entry Criteria ensure that all necessary preparations are complete before testing begins, while **Exit Criteria** ensure that testing is completed successfully and the software is ready for release. Together, they improve the efficiency and quality of the testing process.

Q3 (c) Analyze Test Policy & Test Strategy Included in Test Documentation. [7 Marks]

Definition

Test Documentation is a collection of documents prepared during the software testing process. It helps in planning, executing, monitoring, and controlling testing activities. Two important documents are **Test Policy** and **Test Strategy**.

1. Test Policy

Definition

A **Test Policy** is a **high-level document** prepared by the organization that defines the overall goals, principles, standards, and objectives of software testing. It applies to all projects in the organization.

Contents of Test Policy

1. Testing objectives and goals.
2. Quality standards to be followed.
3. Roles and responsibilities of the testing team.
4. Testing processes and guidelines.
5. Risk management approach.
6. Defect reporting and management policy.
7. Tools and resources to be used.

Advantages

- Provides a common direction for all testing activities.
 - Ensures consistency across projects.
 - Helps maintain software quality.
 - Supports compliance with organizational standards.
-

2. Test Strategy

Definition

A **Test Strategy** is a **project-specific document** that describes **how testing will be performed** for a particular software project.

Contents of Test Strategy

1. Scope of testing.
2. Types of testing (Unit, Integration, System, Acceptance).
3. Testing approach (Manual/Automation).
4. Test environment and test data.
5. Entry and Exit Criteria.
6. Roles and responsibilities.
7. Test schedule and deliverables.
8. Risk analysis and mitigation.

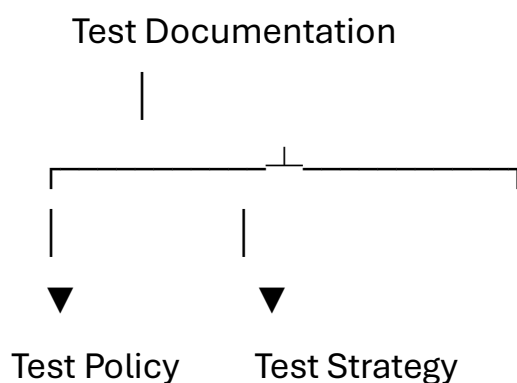
Advantages

- Defines a clear testing approach.
- Improves planning and resource utilization.
- Reduces testing risks.
- Ensures complete test coverage.

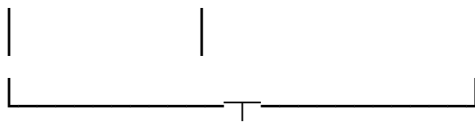
Difference Between Test Policy and Test Strategy

Test Policy	Test Strategy
Organization-level document	Project-level document
Defines overall testing principles	Defines testing approach for a project
Applicable to all projects	Specific to one project
Prepared by senior management	Prepared by Test Manager/Test Lead
Long-term document	Created for each project
Focuses on goals and standards	Focuses on execution and implementation

Relationship Diagram



(What & Why) (How, When & Who)



Test Execution

Conclusion

Test Policy provides the overall direction and quality standards for software testing, whereas **Test Strategy** defines the detailed approach for testing a specific project. Together, they form an essential part of **test documentation** and ensure systematic, efficient, and high-quality software testing.

Q4 (a) Explain Use Case Testing with One Example. [7 Marks]

Definition

Use Case Testing is a **black-box testing technique** in which test cases are designed based on the **use cases** of the system. It verifies whether the software performs correctly according to the user's interactions and functional requirements.

A **use case** describes how a user interacts with the system to achieve a specific goal.

Steps in Use Case Testing

1. Identify the actor (user).
2. Identify the use case (function to be performed).
3. Define the preconditions.
4. Identify the main flow (normal scenario).
5. Identify alternate or exception flows.
6. Design test cases based on each flow.
7. Execute the test cases and compare actual results with expected results.

Example: ATM Cash Withdrawal

Actor

- Customer

Use Case

- Withdraw Cash

Preconditions

- ATM card is valid.
- User has sufficient account balance.
- ATM is connected to the bank server.

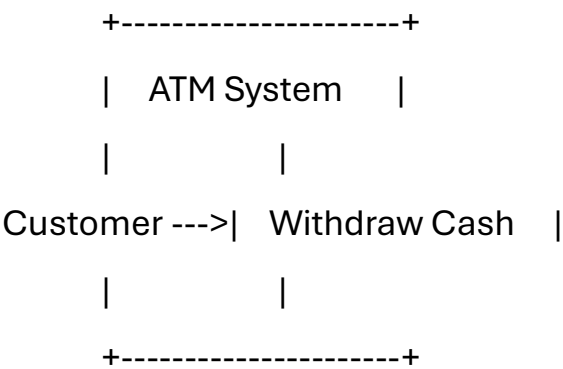
Main Flow

1. Insert ATM card.
2. Enter PIN.
3. Select **Withdraw Cash**.
4. Enter withdrawal amount.
5. Confirm transaction.
6. Cash is dispensed.
7. Receipt is printed and transaction is completed.

Alternate Flow

- Incorrect PIN → Display "**Invalid PIN**".
- Insufficient balance → Display "**Insufficient Balance**".
- ATM out of cash → Display "**Transaction Cannot Be Completed**".

Use Case Diagram



Sample Test Cases

Test Case ID	Test Scenario	Expected Result
TC01	Valid card, correct PIN, sufficient balance	Cash is dispensed successfully
TC02	Invalid PIN	Error message displayed
TC03	Insufficient balance	Transaction rejected with proper message
TC04	ATM has no cash	Transaction cancelled with error message

Advantages

- Covers complete user requirements.
 - Easy to design test cases from use cases.
 - Identifies functional defects.
 - Improves requirement coverage.
 - Ensures the system behaves correctly from the user's perspective.
-

Disadvantages

- Depends on accurate and complete use case documentation.
 - Does not test internal code logic.
 - Complex systems may require many use cases.
-

Conclusion

Use Case Testing verifies that the software performs correctly according to real user interactions. By testing both normal and alternative scenarios, it ensures that the application meets functional requirements and provides a reliable user experience.

Q4 (a) Explain Use Case Testing with One Example. [7 Marks]

Definition

Use Case Testing is a **black-box testing technique** in which test cases are designed based on the **use cases** of the system. It verifies whether the software performs correctly according to the user's interactions and functional requirements.

A **use case** describes how a user interacts with the system to achieve a specific goal.

Steps in Use Case Testing

1. Identify the actor (user).
 2. Identify the use case (function to be performed).
 3. Define the preconditions.
 4. Identify the main flow (normal scenario).
 5. Identify alternate or exception flows.
 6. Design test cases based on each flow.
 7. Execute the test cases and compare actual results with expected results.
-

Example: ATM Cash Withdrawal

Actor

- Customer

Use Case

- Withdraw Cash

Preconditions

- ATM card is valid.
- User has sufficient account balance.
- ATM is connected to the bank server.

Main Flow

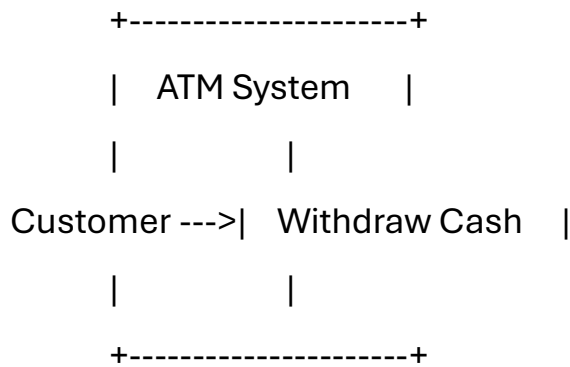
1. Insert ATM card.
2. Enter PIN.
3. Select **Withdraw Cash**.
4. Enter withdrawal amount.
5. Confirm transaction.
6. Cash is dispensed.
7. Receipt is printed and transaction is completed.

Alternate Flow

- Incorrect PIN → Display **"Invalid PIN"**.
- Insufficient balance → Display **"Insufficient Balance"**.

- ATM out of cash → Display "**Transaction Cannot Be Completed**".

Use Case Diagram



Sample Test Cases

Test Case ID	Test Scenario	Expected Result
TC01	Valid card, correct PIN, sufficient balance	Cash is dispensed successfully
TC02	Invalid PIN	Error message displayed
TC03	Insufficient balance	Transaction rejected with proper message
TC04	ATM has no cash	Transaction cancelled with error message

Advantages

- Covers complete user requirements.
- Easy to design test cases from use cases.
- Identifies functional defects.
- Improves requirement coverage.
- Ensures the system behaves correctly from the user's perspective.

Disadvantages

- Depends on accurate and complete use case documentation.
- Does not test internal code logic.

- Complex systems may require many use cases.
-

Conclusion

Use Case Testing verifies that the software performs correctly according to real user interactions. By testing both normal and alternative scenarios, it ensures that the application meets functional requirements and provides a reliable user experience.

Q4 (b) How Can You Classify the Role of Software? [7 Marks]

Definition

Software is a collection of programs, procedures, and related documentation that enables a computer to perform specific tasks. It plays different roles in various fields of technology and daily life.

Classification of the Role of Software

1. System Software

- Controls and manages computer hardware.
- Provides a platform for running application software.

Examples: Windows, Linux, Device Drivers.

2. Application Software

- Performs specific tasks for users.
- Helps users complete daily or business activities.

Examples: MS Word, Excel, Google Chrome.

3. Embedded Software

- Installed inside hardware devices to control their operation.
- Performs dedicated functions.

Examples: ATM, Smart TV, Washing Machine, Car Control System.

4. Web-Based Software

- Runs on web browsers through the Internet.
- Accessible from anywhere.

Examples: Gmail, Amazon, Online Banking.

5. Mobile Software

- Designed for smartphones and tablets.
- Provides mobile-based services.

Examples: WhatsApp, PhonePe, Instagram.

6. Scientific and Engineering Software

- Used for research, simulations, calculations, and engineering applications.

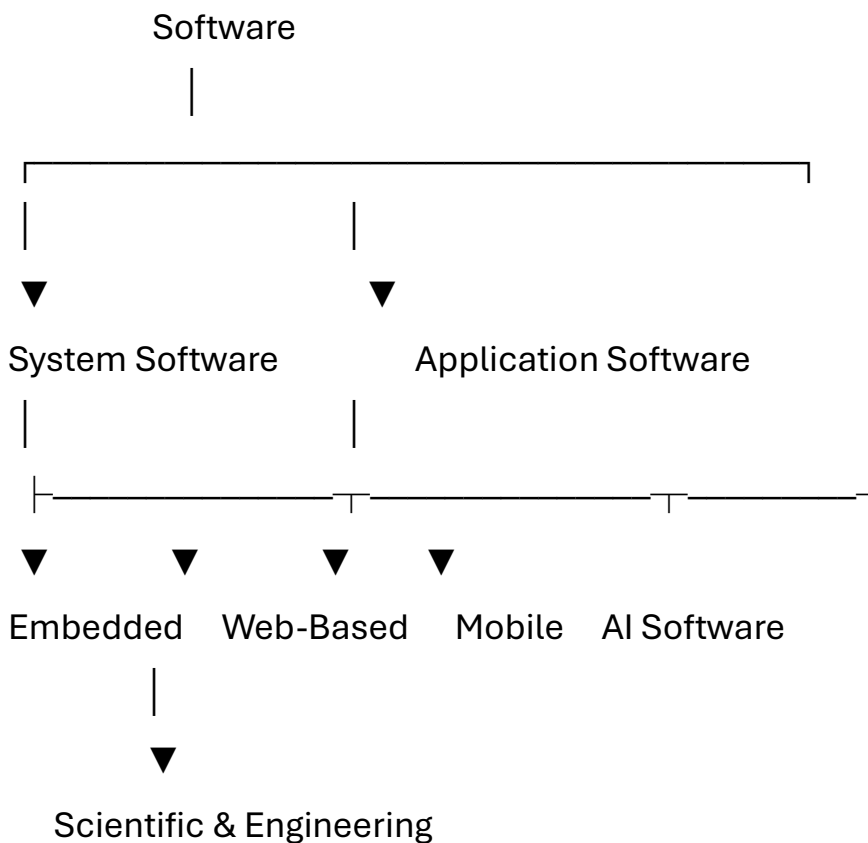
Examples: MATLAB, AutoCAD, ANSYS.

7. Artificial Intelligence Software

- Performs intelligent tasks such as learning, reasoning, and decision-making.

Examples: ChatGPT, Google Assistant, Self-driving car software.

Diagram



Importance of Software

- Automates tasks.
 - Improves productivity.
 - Controls hardware devices.
 - Enables communication.
 - Supports business operations.
 - Helps in research and innovation.
-

Conclusion

Software plays an important role in modern computing. It can be classified as **System Software, Application Software, Embedded Software, Web-Based Software, Mobile Software, Scientific & Engineering Software, and AI Software**, each serving different purposes in various domains.

Q4 (c) Differentiate Between Test Plan and Test Strategy. [7 Marks]

Definition

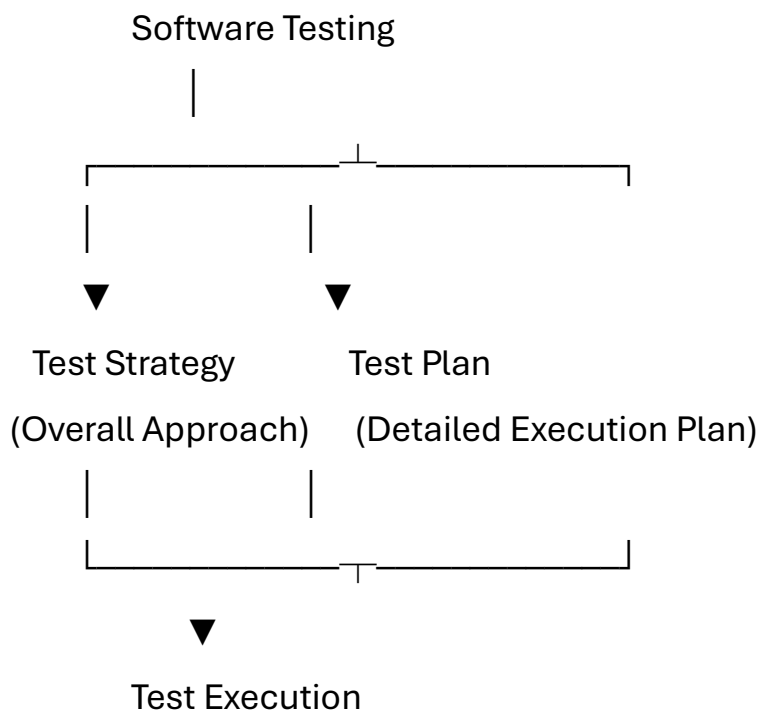
- **Test Plan:** A **Test Plan** is a detailed document that describes **what to test, how to test, when to test, and who will perform the testing** for a specific project.
 - **Test Strategy:** A **Test Strategy** is a high-level document that defines the **overall testing approach, objectives, standards, and methods** to be followed during software testing.
-

Difference Between Test Plan and Test Strategy

Basis	Test Plan	Test Strategy
Definition	Detailed document describing testing activities for a project	High-level document describing the overall testing approach
Purpose	Explains how testing will be executed	Defines the testing methodology and objectives
Scope	Project-specific	Organization-wide or project-level
Prepared By	Test Manager / Test Lead	Project Manager / QA Manager / Senior Management

Basis	Test Plan	Test Strategy
Contents	Test objectives, schedule, resources, test cases, responsibilities	Testing approach, testing types, standards, tools, risk management
Level of Detail	Detailed	High-level
Changes	Can be updated during the project	Rarely changes once approved

Relationship Diagram



Advantages of Test Plan

- Defines testing schedule and responsibilities.
 - Helps manage testing resources.
 - Tracks testing progress.
 - Ensures complete test coverage.
-

Advantages of Test Strategy

- Provides a standard testing approach.
- Improves consistency across projects.
- Reduces testing risks.

- Ensures quality objectives are achieved.
-

Conclusion

A **Test Strategy** defines the **overall approach** and guidelines for testing, while a **Test Plan** provides the **detailed execution plan** for a specific project. Both documents are essential for effective software testing and successful project completion.

Q3 (a) Explain Types of Test Artifacts. [7 Marks]

Definition

Test Artifacts are the **documents, reports, and deliverables** created during the software testing life cycle (STLC). They help in planning, executing, monitoring, and documenting testing activities.

Types of Test Artifacts

1. Test Plan

- A document that describes the **scope, objectives, schedule, resources, and approach** for testing.
 - It answers **what, when, who, and how** to test.
-

2. Test Strategy

- A high-level document that defines the **overall testing approach**.
 - It includes testing types, tools, techniques, and risk management.
-

3. Test Case

- A document containing **test steps, input data, expected results, and actual results**.
 - Used to verify whether a specific function works correctly.
-

4. Test Data

- The data used to execute test cases.
- Includes **valid, invalid, boundary, and null values**.

Example:

- Username: admin
- Password: admin123

5. Defect (Bug) Report

- Records defects found during testing.
- Contains defect ID, description, severity, priority, status, and assigned developer.

6. Test Execution Report

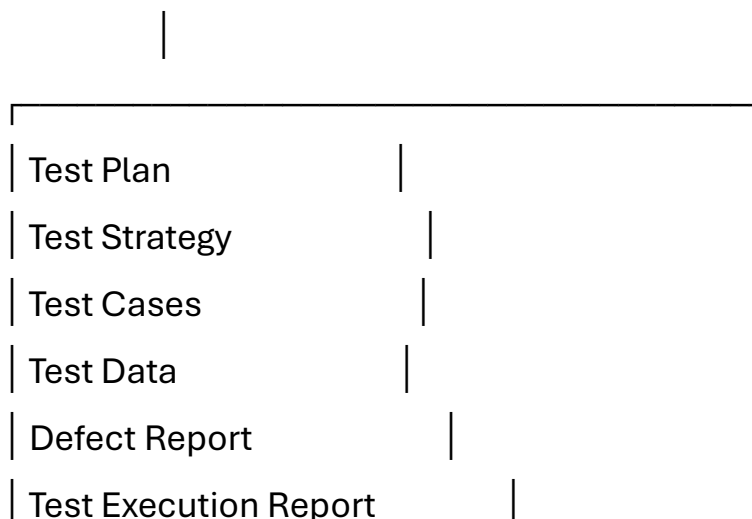
- Summarizes the execution of test cases.
- Includes:
 - Total test cases executed
 - Passed
 - Failed
 - Blocked
 - Pending

7. Test Summary Report

- Final report prepared after testing is completed.
- Contains overall testing results, defect summary, test coverage, and release recommendation.

Types of Test Artifacts Diagram

Test Artifacts



Importance of Test Artifacts

- Helps in planning and managing testing.
 - Provides complete documentation of testing activities.
 - Improves communication among team members.
 - Tracks defects effectively.
 - Ensures better software quality.
-

Conclusion

Test Artifacts are essential documents created throughout the testing process. They help organize testing activities, monitor progress, manage defects, and ensure that the software meets the required quality standards.

Q3 (b) Justify:

(i) Green Money – Cost of Prevention

(ii) Red Money – Cost of Failure [7 Marks]

Definition

In **Software Quality Management**, the **Cost of Quality (COQ)** is the total cost incurred to achieve and maintain software quality. It is broadly divided into:

- **Green Money → Cost of Prevention** (Money spent to prevent defects.)
 - **Red Money → Cost of Failure** (Money spent because defects occur.)
-

(i) Green Money – Cost of Prevention

Definition

Green Money refers to the **cost spent on preventing defects before they occur** during software development. Investing in prevention reduces future failures and maintenance costs.

Activities Included

- Employee training
- Requirement analysis

- Test planning
- Code reviews
- Design reviews
- Quality assurance activities
- Process improvement

Advantages

- Reduces software defects.
 - Improves software quality.
 - Lowers maintenance cost.
 - Increases customer satisfaction.
 - Saves time and resources.
-

(ii) Red Money – Cost of Failure

Definition

Red Money refers to the **cost incurred due to defects found in the software**. It includes the cost of correcting errors after they occur.

Activities Included

- Bug fixing
- Re-testing
- Maintenance
- Customer support
- Warranty claims
- Software rework
- Loss of customer trust

Disadvantages

- Increases development cost.
 - Delays software release.
 - Reduces customer satisfaction.
 - Damages company reputation.
 - Increases maintenance effort.
-

Difference Between Green Money and Red Money

Green Money (Cost of Prevention)

Money spent to prevent defects

Occurs before defects happen

Improves software quality

Lower overall cost

Examples: Training, Reviews, QA

Red Money (Cost of Failure)

Money spent to fix defects

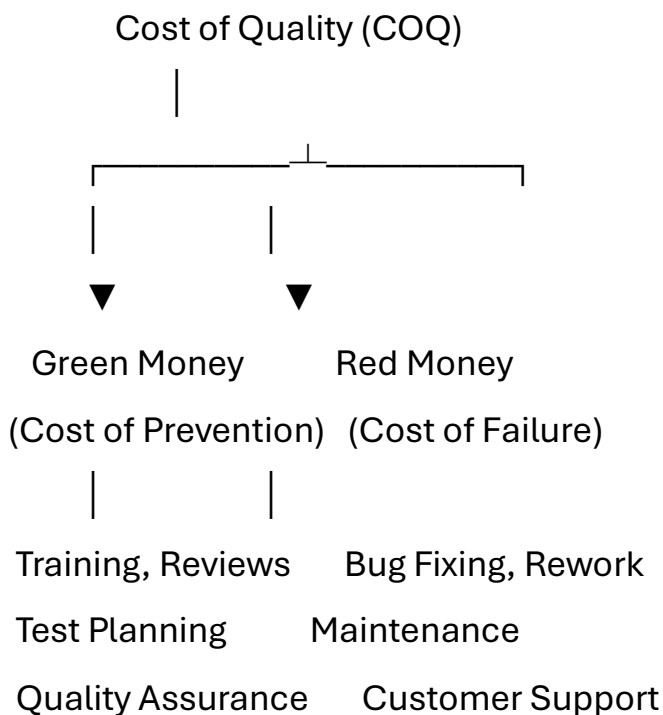
Occurs after defects occur

Reduces software quality if defects are many

Higher overall cost

Examples: Bug fixing, Maintenance, Customer support

Diagram



Conclusion

Green Money (Cost of Prevention) is an investment made to avoid defects and improve software quality, whereas **Red Money (Cost of Failure)** is the expense incurred due to defects after they occur. Organizations should focus more on **Green Money** because prevention is less expensive than correcting failures.

Q3 (c) Discuss Integration Testing and Acceptance Testing. [7 Marks]

Definition

- **Integration Testing** is a testing technique in which **two or more modules are combined and tested together** to verify that they interact correctly.
 - **Acceptance Testing** is the final level of testing performed to verify that the software meets **customer requirements** and is ready for release.
-

1. Integration Testing

Definition

Integration Testing is performed **after Unit Testing** to check whether the integrated modules communicate and exchange data correctly.

Objectives

- Verify interaction between modules.
- Detect interface defects.
- Ensure proper data flow.

Types of Integration Testing

- **Top-Down Integration Testing**
- **Bottom-Up Integration Testing**
- **Big Bang Integration Testing**
- **Hybrid (Sandwich) Integration Testing**

Example

In an **Online Shopping System**, test whether:

- Login → Product Search → Add to Cart → Payment modules work together correctly.

Advantages

- Detects interface defects early.
 - Ensures smooth communication between modules.
 - Improves overall system reliability.
-

2. Acceptance Testing

Definition

Acceptance Testing is the **last phase of testing**, performed by the **customer or end user** to verify that the software satisfies business and user requirements.

Objectives

- Validate software against customer requirements.
- Decide whether the software is ready for deployment.

Types of Acceptance Testing

- **User Acceptance Testing (UAT)**
- **Alpha Testing**
- **Beta Testing**

Example

A college tests the **Attendance Management System** before officially using it to ensure all required features work correctly.

Advantages

- Confirms customer satisfaction.
- Verifies business requirements.
- Reduces the risk of software rejection.

Difference Between Integration Testing and Acceptance Testing

Integration Testing	Acceptance Testing
Tests interaction between modules	Tests complete software against user requirements
Performed after Unit Testing	Performed after System Testing
Conducted by developers/testers	Conducted by customers or end users
Detects interface defects	Validates business requirements
Focuses on module integration	Focuses on software acceptance

Testing Flow Diagram

Unit Testing



Integration Testing



System Testing



Acceptance Testing



Software Release

Conclusion

Integration Testing ensures that different software modules work together correctly, while **Acceptance Testing** verifies that the complete software meets customer expectations and is ready for release. Both are essential phases for delivering reliable and high-quality software.

Q4 (a) What are the Entry and Exit Criteria of Testing? [7 Marks]

Definition

Entry Criteria and **Exit Criteria** are predefined conditions used in the software testing process.

- **Entry Criteria** are the conditions that must be satisfied **before testing begins**.
- **Exit Criteria** are the conditions that must be satisfied **before testing is completed**.

These criteria ensure that testing starts and ends in a controlled and systematic manner.

Entry Criteria of Testing

The following conditions should be fulfilled before testing starts:

1. Requirement Documents are Approved

- Software Requirement Specification (SRS) is complete and approved.

2. Test Plan is Prepared

- Test plan and testing strategy are ready.

3. Test Cases and Test Data are Ready

- Test cases are reviewed and approved.

- Required test data is available.

4. Test Environment is Set Up

- Hardware, software, database, and network are configured.

5. Stable Software Build is Available

- Developers provide a stable build for testing.

6. Testing Tools are Installed

- Required testing and defect tracking tools are available.

7. Resources are Available

- Testers and required resources are assigned.
-

Exit Criteria of Testing

Testing is considered complete when the following conditions are met:

1. All Test Cases are Executed

- All planned test cases have been run.

2. Critical Defects are Fixed

- High-priority and critical defects are resolved.

3. Regression Testing is Completed

- Verify that changes have not introduced new defects.

4. Test Coverage is Achieved

- Required requirement and test case coverage is completed.

5. Test Report is Prepared

- Test summary report and defect report are completed.

6. Acceptance Criteria are Met

- Software satisfies business and customer requirements.

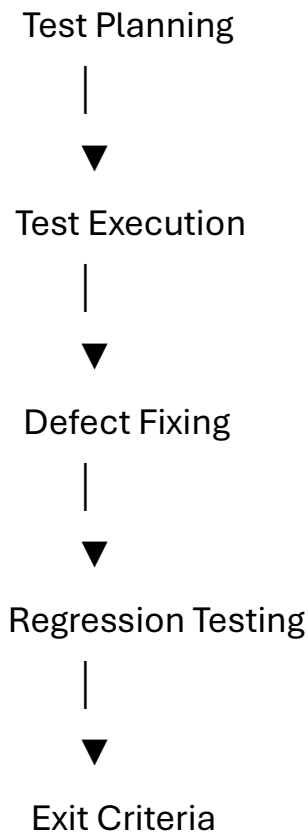
7. Customer/Manager Approval

- Software is approved for release.
-

Diagram

Entry Criteria





Difference Between Entry and Exit Criteria

Entry Criteria

Checked before testing starts

Ensures testing can begin

Includes SRS, test plan, test cases, environment, build

Exit Criteria

Checked before testing ends

Ensures testing can be completed

Includes executed test cases, fixed defects, reports, approval

Advantages

- Ensures systematic testing.
- Improves software quality.
- Reduces testing risks.
- Prevents incomplete testing.
- Helps in timely software release.

Q4 (b) Define and Explain Configuration Management. [7 Marks]

Definition

Configuration Management (CM) is the process of **identifying, organizing, controlling, tracking, and maintaining changes** made to software products throughout the Software Development Life Cycle (SDLC). It ensures that the correct versions of software, documents, and related items are available at all times.

Objectives of Configuration Management

- Control changes in software.
 - Maintain software consistency and integrity.
 - Manage different software versions.
 - Prevent unauthorized changes.
 - Support team collaboration and software maintenance.
-

Activities of Configuration Management

1. Configuration Identification

- Identify all configuration items (CIs) such as source code, documents, test cases, and design files.
 - Assign unique names and version numbers.
-

2. Version Control

- Maintain multiple versions of software.
- Keep track of changes made by developers.

Example: Git, GitHub.

3. Change Control

- Review, approve, and implement software changes.
 - Ensure only authorized changes are applied.
-

4. Configuration Status Accounting

- Record and report the status of configuration items.
 - Track changes, versions, and approvals.
-

5. Configuration Audit

- Verify that software and documents are correct, complete, and follow standards.
 - Ensure approved changes are implemented properly.
-

Configuration Management Process

Configuration Identification

|



Version Control

|



Change Control

|



Configuration Status Accounting

|



Configuration Audit

Advantages

- Maintains software consistency.
 - Controls software versions.
 - Reduces development errors.
 - Improves teamwork.
 - Simplifies maintenance and updates.
 - Supports software recovery if problems occur.
-

Applications

- Banking Software
- Hospital Management System
- Military Software
- Online Shopping Systems

- Mobile Applications

Q4 (c) Differentiate Between Verification and Validation. [7 Marks]

Definition

Verification

Verification is the process of checking whether the software is developed **according to the specified requirements and design**.

Simple Definition:

Verification checks **"Are we building the product correctly?"**

Validation

Validation is the process of checking whether the developed software **meets customer requirements and works correctly**.

Simple Definition:

Validation checks **"Are we building the right product?"**

Difference Between Verification and Validation

Verification

Checks whether the software is built correctly.

Performed during the development process.

Does **not** execute the software.

Finds errors in documents, design, and code.

Performed by developers, reviewers, and QA team.

Includes reviews, inspections, and walkthroughs.

Prevents defects.

Validation

Checks whether the right software is built.

Performed after the software is developed.

Executes the software to test its functionality.

Finds defects in the working software.

Performed by testers and end users.

Includes Unit, Integration, System, and Acceptance Testing.

Detects defects.

Diagram

Software Development

Verification

(Are we building the product correctly?)



Software Developed



Validation

(Are we building the right product?)

Advantages

Verification

- Detects errors early.
- Reduces development cost.
- Improves software quality.

Validation

- Ensures customer satisfaction.
- Verifies software functionality.
- Confirms software is ready for release.

Conclusion

Verification ensures that the software is developed correctly according to specifications, while **Validation** ensures that the final software meets customer needs. Both are important for developing **high-quality and reliable software**.

Q3 (a) Explain Test Efficiency and Defect Rejection. [7 Marks]

Definition

Test Efficiency

Test Efficiency is a metric that measures **how effectively the testing process finds defects** in the software.

Simple Definition:

It shows **how good the testing team is at finding bugs**.

Test Efficiency

Formula

$$\text{Test Efficiency} = \frac{\text{Number of Defects Found During Testing}}{\text{Total Number of Defects}} \times 100$$

Explanation

- Higher test efficiency means the testing process is more effective.
- It indicates that most defects are found before software release.

Example

- Defects found during testing = **90**
- Total defects = **100**

Test Efficiency = **90%**

Advantages

- Measures tester performance.
 - Improves software quality.
 - Reduces defects after release.
 - Helps improve the testing process.
-

Defect Rejection

Definition

Defect Rejection is the process of **rejecting a reported defect** when it is found to be **invalid, duplicate, not reproducible, or not actually a software bug**.

Reasons for Defect Rejection

1. Duplicate defect.
2. Not reproducible.
3. Works as per requirement.
4. Configuration or environment issue.

- 5. User misunderstanding.
- 6. Enhancement request (not a bug).

Example

A tester reports:

"Login page background color is blue instead of white."

The requirement document specifies that the background should be **blue**.

Hence, the developer **rejects** the defect because it is **not actually a defect**.

Difference Between Test Efficiency and Defect Rejection

Test Efficiency

Measures effectiveness of testing.

Calculated using testing metrics.

Higher value indicates better testing.

Focuses on defect detection.

Defect Rejection

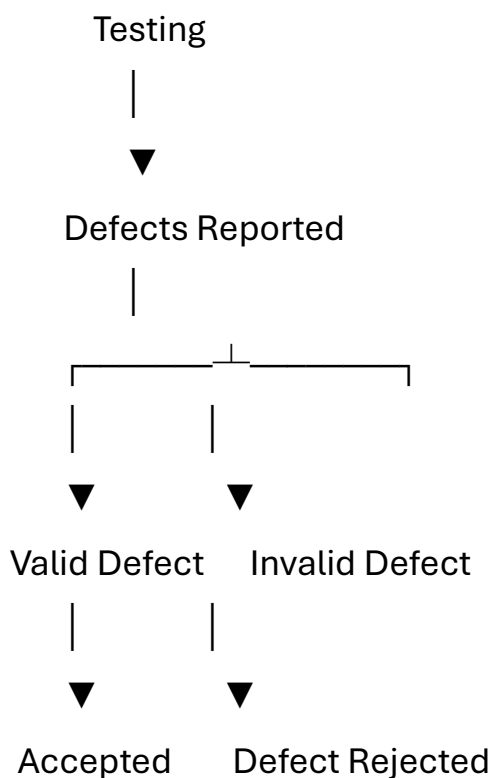
Rejects invalid defect reports.

Based on defect verification.

Reduces unnecessary defect fixing.

Focuses on defect validation.

Diagram



Conclusion

Test Efficiency measures how effectively testers identify defects, while **Defect Rejection** ensures that only genuine defects are accepted for fixing. Both are important metrics for improving software quality and the testing process.

Q3 (b) What are the Entry & Exit Criteria of Testing? [7 Marks]

Definition

Entry Criteria and **Exit Criteria** are the conditions that determine **when testing should start and when it should end**.

- **Entry Criteria:** Conditions that must be completed **before starting testing**.
 - **Exit Criteria:** Conditions that must be completed **before finishing testing**.
-

Entry Criteria of Testing

Before testing starts, the following conditions should be satisfied:

1. Requirement documents are approved.

- Software Requirement Specification (SRS) is complete.

2. Test Plan is ready.

- Test plan and test strategy are prepared.

3. Test Cases and Test Data are ready.

- Test cases are reviewed and approved.

4. Test Environment is available.

- Hardware, software, and network are properly configured.

5. Stable Software Build is available.

- Developers provide a stable version of the software for testing.

6. Testing tools are installed.

- Required testing and defect tracking tools are available.

7. Testing team is available.

- Testers and necessary resources are assigned.
-

Exit Criteria of Testing

Testing is completed when the following conditions are met:

1. All test cases are executed.

- Planned test cases have been completed.

2. Critical defects are fixed.

- High-priority bugs are resolved.

3. Regression testing is completed.

- Ensure that recent changes have not introduced new defects.

4. Test coverage is achieved.

- Required test case and requirement coverage is completed.

5. Test report is prepared.

- Test summary report and defect report are completed.

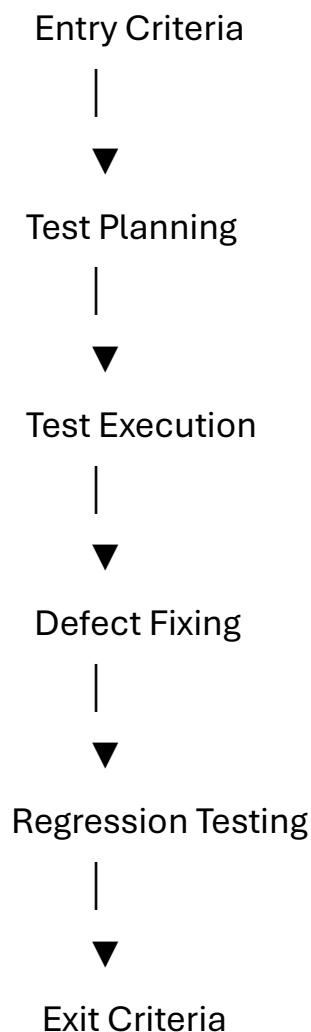
6. Customer requirements are satisfied.

- Software meets business and user requirements.

7. Approval for release is obtained.

- Project manager or customer approves the software.

Diagram



Difference Between Entry & Exit Criteria

Entry Criteria	Exit Criteria
Checked before testing starts	Checked before testing ends
Ensures testing can begin	Ensures testing can be completed
Includes SRS, test plan, test cases, environment	Includes executed test cases, fixed defects, reports, and approval

Conclusion

Entry Criteria ensure that all prerequisites are completed before testing starts, whereas **Exit Criteria** ensure that testing is completed successfully and the software is ready for release.

Q3 (c) Describe Cultural Changes Required for Quality Improvement. [7 Marks]

Definition

Cultural Change means changing the **attitude, behavior, and work practices** of employees and management to achieve **continuous quality improvement**. It is an important principle of **Total Quality Management (TQM)**.

Cultural Changes Required for Quality Improvement

1. Customer Focus

- Understand customer needs and provide high-quality software.
 - Customer satisfaction should be the main objective.
-

2. Employee Involvement

- Every employee should participate in quality improvement.
 - Encourage teamwork and responsibility.
-

3. Continuous Improvement

- Improve products, processes, and services regularly.
 - Use the **PDCA (Plan–Do–Check–Act)** cycle for improvement.
-

4. Training and Skill Development

- Provide regular training to employees.
 - Improve technical and problem-solving skills.
-

5. Effective Communication

- Maintain clear communication among developers, testers, managers, and customers.
 - Reduce misunderstandings and errors.
-

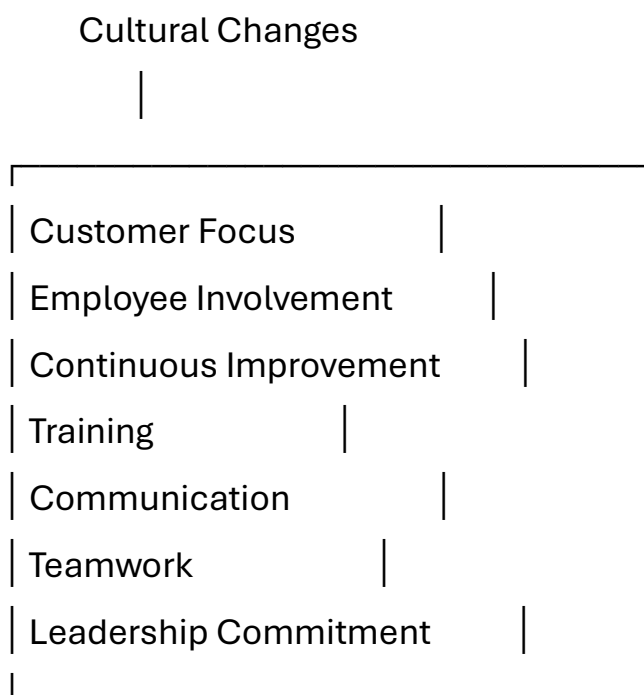
6. Teamwork and Collaboration

- Encourage cooperation among all departments.
 - Share knowledge and work together to achieve quality goals.
-

7. Leadership Commitment

- Management should support quality initiatives.
 - Leaders should motivate employees and provide the necessary resources.
-

Diagram



Benefits of Cultural Changes

- Improves software quality.
 - Reduces defects.
 - Increases productivity.
 - Improves teamwork.
 - Enhances customer satisfaction.
 - Supports continuous improvement.
-

Conclusion

Cultural changes help create a **quality-oriented work environment**. By focusing on **customer satisfaction, teamwork, employee involvement, training, leadership, and continuous improvement**, organizations can produce reliable and high-quality software.

Q3 (c) Describe Quality Assurance (QA) Process. [7 Marks]

Definition

Quality Assurance (QA) is a **planned and systematic process** that ensures software is developed according to **specified quality standards** and customer requirements. QA focuses on **preventing defects** rather than finding them.

Quality Assurance Process

1. Requirement Analysis

- Study and verify customer requirements.
 - Ensure requirements are complete and clear.
-

2. Quality Planning

- Define quality standards, objectives, and procedures.
 - Prepare the Quality Assurance Plan.
-

3. Process Implementation

- Follow the defined software development process and coding standards.
 - Ensure all team members follow the quality guidelines.
-

4. Reviews and Inspections

- Review requirements, design, code, and documents.
 - Detect defects early before testing.
-

5. Testing

- Perform Unit, Integration, System, and Acceptance Testing.
 - Verify that the software meets quality standards.
-

6. Defect Reporting and Correction

- Record defects found during reviews or testing.
 - Developers fix the defects, and testers verify the fixes.
-

7. Continuous Improvement

- Analyze feedback and quality metrics.
 - Improve the development and testing process using the **PDCA (Plan–Do–Check–Act)** cycle.
-

Quality Assurance Process Diagram

Requirement Analysis



Quality Planning



Process Implementation



Reviews & Inspections



Testing





Defect Correction



Continuous Improvement

Advantages of Quality Assurance

- Prevents defects.
- Improves software quality.
- Reduces development and maintenance costs.
- Increases customer satisfaction.
- Ensures compliance with quality standards.

Conclusion

The **Quality Assurance Process** helps organizations develop **high-quality, reliable, and defect-free software** by following proper planning, reviews, testing, defect correction, and continuous improvement.

Q3 (a) Explain the Role of Test Manager with Example. [7 Marks]

Definition

A **Test Manager** is the person responsible for **planning, managing, monitoring, and controlling all software testing activities**. The Test Manager ensures that the software is tested properly and delivered with high quality.

Roles and Responsibilities of a Test Manager

1. Test Planning

- Prepare the **Test Plan** and define the testing objectives.
- Decide the testing scope, schedule, and resources.

2. Test Strategy

- Select the appropriate testing approach.
- Decide whether testing will be manual, automated, or both.

3. Team Management

- Assign testing tasks to team members.
- Guide and support testers during testing.

4. Test Monitoring

- Monitor testing progress.
- Ensure testing is completed on time.

5. Defect Management

- Track reported defects.
- Coordinate with developers to fix bugs.

6. Risk Management

- Identify testing risks.
- Take necessary actions to reduce project risks.

7. Test Reporting

- Prepare test summary reports.
- Report testing status to project managers and clients.

Example

Online Banking System

The **Test Manager**:

- Prepares the test plan for login, fund transfer, and account management.
- Assigns testing tasks to testers.
- Reviews defect reports.
- Ensures all critical defects are fixed before software release.
- Submits the final test report to the project manager.

Role of Test Manager Diagram

Test Manager

|

Test Planning	
Test Strategy	
Team Management	
Test Monitoring	
Defect Management	
Risk Management	
Test Reporting	

Advantages

- Ensures proper test planning.
- Improves software quality.
- Reduces project risks.
- Manages the testing team effectively.
- Ensures software is delivered on time.

Q3 (b) Discuss Unit Testing and Integration Testing. [7 Marks]

Definition

- **Unit Testing** is the process of testing **individual modules or components** of a software application.
- **Integration Testing** is the process of testing **two or more modules together** to ensure they work correctly as a group.

1. Unit Testing

Definition

Unit Testing checks whether a **single module or function** works correctly. It is the **first level of testing** and is usually performed by developers.

Objectives

- Verify each module independently.

- Detect coding errors at an early stage.
- Ensure individual functions work correctly.

Example

In a **Calculator Application**, test the **Addition** function separately.

Input: 10 + 20

Expected Output: 30

Advantages

- Finds bugs early.
 - Easy to debug.
 - Improves code quality.
 - Reduces development cost.
-

2. Integration Testing

Definition

Integration Testing checks whether **multiple modules work together correctly** after unit testing.

Objectives

- Verify communication between modules.
- Detect interface and data flow errors.
- Ensure integrated modules function properly.

Example

In an **Online Shopping System**, test whether:

Login → Product Search → Add to Cart → Payment

work together without errors.

Advantages

- Detects interface defects.
 - Verifies module interaction.
 - Improves system reliability.
-

Difference Between Unit Testing and Integration Testing

Unit Testing

Tests a single module

Performed first

Performed by developers

Detects coding errors

Focuses on individual functions

Integration Testing

Tests multiple integrated modules

Performed after Unit Testing

Performed by testers/developers

Detects interface and communication errors

Focuses on interaction between modules

Diagram

Software Testing

Unit Testing



Integration Testing



System Testing

Conclusion

Unit Testing verifies that each individual module works correctly, while **Integration Testing** ensures that different modules interact properly. Together, they help in developing reliable and high-quality software.

Q4 (a) Explain Test Plan Development Process. [7 Marks]

Definition

A **Test Plan** is a document that describes **what to test, how to test, when to test, who will test, and the resources required** for software testing.

Test Plan Development Process

1. Study Requirements

- Understand the Software Requirement Specification (SRS).

- Identify the testing scope.

2. Define Test Objectives

- Decide what needs to be tested.
- Set testing goals.

3. Select Test Strategy

- Choose manual or automated testing.
- Decide the types of testing to be performed.

4. Prepare Test Cases & Test Data

- Write test cases.
- Prepare valid and invalid test data.

5. Allocate Resources

- Assign testers, tools, hardware, and software.

6. Prepare Test Schedule

- Define start date, end date, and milestones.

7. Review and Approve Test Plan

- Review the plan.
- Get approval from the Test Manager or Project Manager.

Diagram

Requirement Analysis



Define Test Objectives



Select Test Strategy



Prepare Test Cases



Allocate Resources



Prepare Schedule



Review & Approval

Conclusion

A good **Test Plan** helps complete testing on time, improves quality, and ensures all requirements are tested.

Q4 (b) Discuss the Process to Develop Test Strategy. [7 Marks]

Definition

A **Test Strategy** is a high-level document that defines the **overall approach and method** for software testing.

Process to Develop Test Strategy

1. Study Project Requirements

- Understand customer and business requirements.

2. Define Testing Scope

- Decide which features will and will not be tested.

3. Select Testing Types

- Decide Unit, Integration, System, Acceptance, Performance, Security testing, etc.

4. Select Testing Approach

- Decide Manual Testing, Automation Testing, or both.

5. Identify Test Environment

- Specify hardware, software, database, and network requirements.

6. Define Entry & Exit Criteria

- Decide when testing will start and when it will finish.

7. Risk Analysis and Documentation

- Identify testing risks.

- Prepare and approve the Test Strategy document.

Diagram

Project Requirements



Define Scope



Select Testing Types



Choose Testing Approach



Identify Test Environment



Entry & Exit Criteria



Risk Analysis & Documentation

Conclusion

A **Test Strategy** provides a clear roadmap for testing and ensures effective and systematic software testing.

Q4 (c) Write Test Cases for Bank ATM. [7 Marks]

Definition

A **Test Case** is a document that specifies the **input, execution steps, and expected result** to verify a software function.

Module: ATM Cash Withdrawal

Test Case ID	Test Scenario	Input	Expected Result
TC01	Valid ATM card and correct PIN	Valid Card, Correct PIN	Login successful
TC02	Invalid PIN	Valid Card, Wrong PIN	Display "Invalid PIN"
TC03	Expired ATM card	Expired Card	Display "Card Expired"
TC04	Insufficient account balance	Withdraw ₹10,000 (Balance ₹5,000)	Display "Insufficient Balance"
TC05	Withdraw valid amount	Balance ₹20,000, Withdraw ₹5,000	Cash dispensed successfully
TC06	ATM has no cash	Valid transaction	Display "ATM Out of Cash"
TC07	Cancel transaction	Press Cancel	Transaction cancelled and card returned
TC08	Card not inserted properly	Damaged/Incorrect insertion	Display "Insert Card Properly"

ATM Transaction Flow

Insert ATM Card



Enter PIN



Correct?



No Yes



Error Select Transaction



Enter Amount





Sufficient Balance?



No

Yes



Error Message Cash Dispensed

Conclusion

These test cases verify that the ATM system correctly handles **valid transactions, invalid inputs, insufficient balance, and error conditions**, ensuring reliable and secure operation.

Based on:

-  **SPPU STQA Unit 2 syllabus**
-  **All PYQs you shared**
-  **SPPU question pattern (last 5–6 papers)**

These are the **most likely NEW questions** that can appear from **Unit 2**.

★★★★★ **Very High Probability (Must Prepare)**

1. Explain Test Metrics. [7]

- Definition
- Types
- Importance
- Examples

2. Explain Test Case Productivity. [7]

- Definition
- Formula
- Advantages

- Example
-

3. Explain Test Case Coverage. [7]

- Definition
 - Formula
 - Importance
 - Benefits
-

4. Explain Test Monitoring and Control. [7]

- Definition
 - Activities
 - Benefits
-

5. Explain Scenario Testing with Example. [7]

(Use Case Testing has already been asked.)

6. Explain Role of Tester. [7]

(Role of Test Manager already asked.)

7. Explain Test Report. [7]

- Purpose
 - Contents
 - Advantages
-

8. Explain Documentation Risks and Issues. [7]

Direct syllabus topic.

9. Explain Quality Assurance Process. [7]

Very important.

10. Explain Quality Best Practices. [7]

Direct syllabus topic.

★★★★ High Probability

11. Explain Test Execution Schedule. [7]

12. Explain Test Data with Example. [7]

(Test Case already asked.)

13. Explain Test Organization. [7]

- Test Manager
 - Tester
 - Responsibilities
-

14. Explain Defect Acceptance. [7]

(Defect Rejection already asked.)

15. Explain Test Planning Artifacts. [7]

(Different from Types of Test Artifacts.)

16. Explain Quality Management Importance. [7]

Direct syllabus.

17. Explain Test Strategy and Test Approach. [7]

18. Explain Test Plan Contents. [7]

Purpose + Contents.

★★★ Medium Probability

19. Differentiate Test Case and Test Scenario.

20. Differentiate Test Plan and Test Case.

21. Differentiate Test Strategy and Test Approach.

22. Explain Test Entry and Exit Criteria with Example.

23. Explain Effort Variance and Schedule Variance.

Direct syllabus.

24. Explain Test Effort Biasing Factors.

One of the few untouched syllabus topics.

★ ★ Application-Based Questions (Very Likely)

Write Test Cases for

- Student Registration System
 - Railway Reservation System
 - Hospital Management System
 - Online Shopping Cart
 - Online Banking
 - College Attendance
 - Library Management System
-

Prepare Test Plan for

- ATM System
 - Online Shopping
 - Railway Reservation
 - Hospital Management
 - College Attendance
-

Prepare Test Strategy for

- Banking Software
 - E-commerce Website
 - Library Management
 - Student Portal
-

Top 10 Predictions (Highest Chance)

1. Test Metrics ★★★★★
2. Test Monitoring & Control ★★★★★
3. Scenario Testing ★★★★★
4. Test Case Coverage ★★★★★
5. Test Case Productivity ★★★★★
6. Test Report ★★★★★
7. Role of Tester ★★★★★
8. Documentation Risks & Issues ★★★★★
9. Test Execution Schedule ★★★★★
10. Test Effort Biasing Factors ★★★★★

Expected Coverage

If you prepare:

- **All the PYQs you've already collected, and**
- **These 10 predicted questions,**

you'll likely cover **90–95% of the concepts** that SPPU is likely to test from **Unit 2**